

Checklist:

Team Topologies – Conway's Law

1. Clarify the Desired Software Architecture

- Have we defined the software architecture we actually want?
- Are we avoiding designing software separately from team design?
- Do team boundaries support the desired system boundaries?
- Are services, APIs, databases, and interfaces aligned with team ownership?
- Are we designing for fast flow of change, not just technical elegance?

2. Check for Conway's Law Effects

- Does our current organization structure naturally produce the architecture we want?
- Are communication paths between teams reflected in the software structure?
- Are shared teams, shared databases, or shared tools creating unwanted coupling?
- Are team dependencies slowing delivery?
- Are we asking: "Is there a better design unavailable to us because of our organization?"

3. Use the Reverse Conway Maneuver

- Have we shaped team boundaries to match the architecture we want?
- Are teams arranged around business capabilities or product areas?
- Can teams deliver changes from design through deployment with minimal handoffs?
- Have we reduced reliance on centralized specialist teams where possible?
- Are teams given enough autonomy to own their part of the system?

4. Enable Team-Scoped Flow

- Does each team have a clear, bounded responsibility?
- Are components loosely coupled?
- Are components highly cohesive?
- Are version compatibility expectations clear?
- Is cross-team testing clear and appropriate?
- Can teams make changes safely without excessive coordination?

5. Involve Technical Expertise in Organization Design

- Are technical leaders involved in team-structure decisions?
- Do managers understand how team design affects system architecture?
- Are service ownership and team responsibilities decided with architectural input?
- Are architects considering both technical and social/organizational factors?
- Are team boundaries being designed intentionally rather than by budget or politics?

6. Restrict Unnecessary Communication

- Are we avoiding the assumption that more communication is always better?
- Are communication paths focused between the right teams?
- Are teams communicating because of real dependencies, not poor architecture?
- Could better APIs, platforms, or boundaries reduce the need for coordination?
- Are we minimizing unnecessary team-to-team communication paths?

7. Review Tool Choices

- Do shared tools support collaboration where collaboration is actually needed?
- Are separate tools used where clear team boundaries are needed?
- Are tools accidentally forcing teams to interact in unhealthy ways?
- Do teams have access to the operational information they need?
- Are security controls preserved without blocking useful visibility?

8. Avoid Naive Uses of Conway's Law

- Are we avoiding too many small component teams?
- Are stream-aligned teams preferred where fast flow is needed?
- Are complicated-subsystem teams used only when deep expertise is truly required?

- Are we avoiding reorganizations just to reduce headcount or create management fiefdoms?
- Are reorganizations evaluated for their impact on delivery, architecture, and team cognitive load?

9. Final Readiness Check

- Team structure and software architecture support each other.
- Dependencies between teams are intentional and limited.
- Teams can deliver value with minimal handoffs.
- Communication is focused, not chaotic.
- Tooling reinforces the desired collaboration patterns.
- Technical leaders are involved in organization design.
- The organization is designed to improve software flow, not just reporting lines.